

Live Weather Dashboard

Real-time weather using the Fetch API, async/await and dynamic DOM rendering

Internship Project Report

Intern name	_____
Organisation	_____
Mentor / Guide	_____
Duration	_____
Date	September 2026

Technologies: HTML5, CSS3, JavaScript (ES2020), Fetch API, OpenWeatherMap REST API

1. Project Overview

The Live Weather Dashboard is a single-page web application that lets a user search for any city and instantly see its current temperature, a weather icon, and a 5-day forecast. All data is fetched at runtime from the OpenWeatherMap REST API using asynchronous JavaScript, and the interface is built dynamically by injecting HTML into the DOM. No frameworks or libraries are used, so the project focuses on core web fundamentals.

2. Objectives

- Understand asynchronous programming: Promises, `async/await`, and `try/catch` error handling.
- Consume a real REST API with the `fetch()` function and parse JSON responses.
- Render data dynamically using DOM manipulation (`innerHTML`, `createElement`, `DocumentFragment`).
- Validate user input and design clear loading, empty and error states.
- Build a responsive, accessible interface with CSS Grid and Flexbox.

3. Key Features

Feature	Description
Real-time temperature	Current temperature, feels-like, humidity and wind speed for the searched city.
Weather icons	Official OpenWeatherMap icon codes mapped to image URLs, shown for current and forecast.
5-day forecast	40 three-hour data points reduced to one representative reading per day (closest to 12:00).
Async/await + Promise.all	Current weather and forecast requests run in parallel for faster load.
Input validation	Rejects empty, too-short or invalid characters before any network call is made.
Error states	Distinct messages for 404 (city not found), 401 (bad key), 429 (rate limit), and network failure.
Loading state	Spinner shown while waiting, button disabled to prevent duplicate requests.

4. Tools and Technologies

Layer	Technology
Structure	HTML5 semantic elements, ARIA attributes (role=alert, aria-live)
Styling	CSS3 - Grid, Flexbox, gradients, keyframe animation
Logic	Vanilla JavaScript (ES2020) - modules of functions, no dependencies

Layer	Technology
Data source	OpenWeatherMap: /data/2.5/weather and /data/2.5/forecast
Editor / Testing	VS Code, Live Server extension, Chrome DevTools (Network tab)

5. System Architecture and Data Flow

The application follows a simple pipeline. Each step is a small, single-purpose function which keeps the code easy to test and debug.

Step	Function	Purpose
1	validateCity()	Trim and check user input; show inline error if invalid.
2	getWeather()	Build URLs with encodeURIComponent and call both endpoints with Promise.all.
3	fetchJSON()	Wrap fetch(); convert HTTP status codes into readable Error objects.
4	dailyForecast()	Reduce 40 three-hour entries to 5 daily entries.
5	renderCurrent() / renderForecast()	Inject the current card and forecast tiles into the DOM.
6	setLoading() / showFetchError()	Manage loading and error UI states.

6. Setup Instructions

- Create a free account at openweathermap.org and copy your API key (new keys can take up to two hours to activate).
- Create a folder with three files: `index.html`, `style.css`, `script.js` (source in Section 7).
- Replace `YOUR_OPENWEATHERMAP_API_KEY` in `script.js` with your key.
- Open the folder in VS Code and run with Live Server (or open `index.html` directly in a browser).
- Search for a city such as Bengaluru, London or Tokyo to test.

Security note: a key placed in front-end code is visible to anyone. This is acceptable for a learning project with a free key, but production apps should call the API through a backend proxy.

7. Source Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Live Weather Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>Live Weather Dashboard</h1>
    <form id="search-form" novalidate>
      <input id="city-input" type="text" placeholder="Search city, e.g. Bengaluru"
        autocomplete="off" aria-label="City name">
      <button type="submit">Search</button>
    </form>
    <p id="input-error" class="error" role="alert" hidden></p>

    <div id="status" class="status" aria-live="polite"></div>
    <section id="current" class="card" hidden></section>
    <section id="forecast" class="forecast" hidden></section>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: system-ui, sans-serif;
  background: linear-gradient(135deg, #1e3c72, #2a5298);
  min-height: 100vh; color: #fff; padding: 24px;
}
.app { max-width: 720px; margin: 0 auto; }
h1 { text-align: center; margin-bottom: 20px; }
#search-form { display: flex; gap: 8px; }
#city-input { flex: 1; padding: 12px; border-radius: 8px; border: 2px solid transparent; }
#city-input.invalid { border-color: #ff6b6b; }
button { padding: 12px 20px; border: 0; border-radius: 8px; background: #ffb703;
  font-weight: 700; cursor: pointer; }
.error { color: #ffd6d6; background: rgba(255,0,0,.25); padding: 10px;
  border-radius: 8px; margin-top: 10px; }
.status { text-align: center; margin: 16px 0; }
.spinner { width: 32px; height: 32px; margin: 0 auto; border: 4px solid #fff5;
  border-top-color: #fff; border-radius: 50%; animation: spin 1s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }
.card { background: rgba(255,255,255,.15); border-radius: 16px; padding: 24px;
  text-align: center; margin-top: 20px; }
.card .temp { font-size: 3.5rem; font-weight: 700; }
.card .meta { display: flex; justify-content: center; gap: 24px; margin-top: 12px; }
.forecast { display: grid; grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
  gap: 12px; margin-top: 20px; }
.day { background: rgba(255,255,255,.15); border-radius: 12px; padding: 14px;
  text-align: center; }
.day img { width: 56px; height: 56px; }
```

script.js

```
const API_KEY = "YOUR_OPENWEATHERMAP_API_KEY"; // free key: openweathermap.org
const BASE_URL = "https://api.openweathermap.org/data/2.5";
const ICON_URL = (code) => `https://openweathermap.org/img/wn/${code}@2x.png`;

const form = document.getElementById("search-form");
const input = document.getElementById("city-input");
const inputError = document.getElementById("input-error");
const statusBox = document.getElementById("status");
const currentEl = document.getElementById("current");
const forecastEl = document.getElementById("forecast");

// ----- 1. Input validation -----
function validateCity(value) {
  const city = value.trim();
  if (!city) return { ok: false, msg: "Please enter a city name." };
  if (city.length < 2) return { ok: false, msg: "City name is too short." };
  if (!/^[\\p{L}\\s\\.\\-]+$/.test(city))
    return { ok: false, msg: "Use letters only (no numbers or symbols). " };
  return { ok: true, city };
}

function showInputError(msg) {
  inputError.textContent = msg;
  inputError.hidden = !msg;
  input.classList.toggle("invalid", Boolean(msg));
}

// ----- 2. Async data fetching -----
async function fetchJSON(url) {
  const res = await fetch(url);
  if (!res.ok) {
    if (res.status === 404) throw new Error("City not found. Check the spelling.");
    if (res.status === 401) throw new Error("Invalid API key.");
    if (res.status === 429) throw new Error("Too many requests. Try again later.");
    throw new Error(`Server error (${res.status}).`);
  }
  return res.json();
}

async function getWeather(city) {
  const q = encodeURIComponent(city);
  const urls = [
    `${BASE_URL}/weather?q=${q}&units=metric&appid=${API_KEY}`,
    `${BASE_URL}/forecast?q=${q}&units=metric&appid=${API_KEY}`,
  ];
  // Both requests run in parallel -> faster than awaiting one by one
  const [current, forecast] = await Promise.all(urls.map(fetchJSON));
  return { current, forecast };
}

// ----- 3. Data shaping -----
function dailyForecast(list) {
  // API returns 3-hour steps (40 items). Keep one entry per day (nearest 12:00).
  const byDay = {};
  list.forEach((item) => {
    const [date, time] = item.dt_txt.split(" ");
    const gap = Math.abs(parseInt(time) - 12);
    if (!byDay[date] || gap < byDay[date].gap) byDay[date] = { gap, item };
  });
  return Object.values(byDay).map((d) => d.item).slice(0, 5);
}

// ----- 4. Dynamic DOM injection -----
function renderCurrent(d) {
  const w = d.weather[0];
  currentEl.innerHTML = `
    <h2>${d.name}, ${d.sys.country}</h2>
    
    <div class="temp">${Math.round(d.main.temp)}&deg;C</div>
    <p style="text-transform:capitalize">${w.description}</p>
    <div class="meta">
      <span>Feels like ${Math.round(d.main.feels_like)}&deg;C</span>
      <span>Humidity ${d.main.humidity}%</span>
      <span>Wind ${d.wind.speed} m/s</span>
    </div>
  `;
}
```

```

    </div>`;
    currentEl.hidden = false;
  }

function renderForecast(list) {
  forecastEl.innerHTML = "";
  const frag = document.createDocumentFragment();
  dailyForecast(list).forEach((item) => {
    const day = document.createElement("div");
    day.className = "day";
    const name = new Date(item.dt * 1000)
      .toLocaleDateString("en-US", { weekday: "short", day: "numeric" });
    day.innerHTML = `
      <strong>${name}</strong>
      
      <div>${Math.round(item.main.temp)}&deg;C</div>
      <small>${item.weather[0].main}</small>`;
    frag.appendChild(day);
  });
  forecastEl.appendChild(frag);
  forecastEl.hidden = false;
}

// ----- 5. UI states -----
function setLoading(on) {
  statusBox.innerHTML = on ? '<div class="spinner"></div><p>Fetching weather...</p>' : "";
  form.querySelector("button").disabled = on;
}

function showFetchError(msg) {
  currentEl.hidden = forecastEl.hidden = true;
  statusBox.innerHTML = `<p class="error">${msg}</p>`;
}

// ----- 6. Controller -----
form.addEventListener("submit", async (e) => {
  e.preventDefault();
  const result = validateCity(input.value);
  if (!result.ok) return showInputError(result.msg);
  showInputError("");

  setLoading(true);
  try {
    const { current, forecast } = await getWeather(result.city);
    setLoading(false);
    renderCurrent(current);
    renderForecast(forecast.list);
  } catch (err) {
    setLoading(false);
    const offline = err instanceof TypeError; // fetch rejects on network failure
    showFetchError(offline ? "Network error. Check your connection." : err.message);
  }
});

input.addEventListener("input", () => showInputError(""));

```

8. Code Walkthrough: Key Concepts

Async/await. `fetch()` returns a Promise. Using `await` pauses only the current function, not the browser, so the UI stays responsive while data loads.

Error handling. `fetch()` only rejects on network failure; HTTP errors like 404 still resolve. The code checks `res.ok` manually and throws descriptive errors that are caught by a single `try/catch` in the controller.

Promise.all. The two API calls are independent, so running them together roughly halves the wait time compared with two sequential awaits.

DOM injection. The current-weather card uses template literals with `innerHTML`. The forecast uses `createElement` and a `DocumentFragment` so that all tiles are added in a single reflow.

Validation. A regular expression with the Unicode flag accepts letters from any language, so names such as Zürich or São Paulo pass validation.

9. Test Cases

Input / Condition	Expected Result
Empty input, click Search	Inline message: Please enter a city name. No network call.
"a"	Inline message: City name is too short.
"Paris123"	Inline message about letters only.
"Bengaluru"	Current card and five forecast tiles appear.
"Xyzabc"	City not found error shown; previous results hidden.
Wrong API key	Invalid API key message.
Browser set to offline (DevTools)	Network error message.
Slow 3G throttling	Spinner visible, button disabled until finished.

10. Challenges and Solutions

Challenge	Solution
Forecast endpoint returns 40 entries, not 5 days	Grouped by date and picked the entry closest to 12:00 per day.
fetch() does not throw on 404	Checked res.ok and mapped status codes to messages.
Repeated clicks caused duplicate requests	Disabled the button while loading.
Stale results shown after an error	Hid the weather sections inside showFetchError().

11. Future Enhancements

- Geolocation button using `navigator.geolocation` to load local weather.
- Celsius/Fahrenheit toggle and saved preference in `localStorage`.
- Recent-search history and debounced autocomplete suggestions.
- Hourly forecast chart using `Canvas` or `Chart.js`.
- Dynamic background that changes with weather condition and time of day.
- Backend proxy (Node.js/Express) to hide the API key and cache responses.

12. Learning Outcomes

Through this project I gained hands-on experience with asynchronous JavaScript, REST API integration, JSON parsing, DOM manipulation, input validation, and user-focused error handling. I also learned to structure code into small functions, debug using the browser Network tab, and think about loading, empty and failure states as part of the user experience. These skills transfer directly to building data-driven front-end applications.

13. References

- MDN Web Docs - Using the Fetch API, async function, Promise.all
- OpenWeatherMap API documentation - Current weather data and 5 day / 3 hour forecast
- web.dev - Accessible forms and ARIA live regions